NPU-Native Neural Networks on AMD XDNA 2

Hugo Penedones

Conveyor-belt residual inference for HIGGS on AMD XDNA 2

Abstract

This paper studies a deliberately narrow question: what neural-network shape matches the AMD XDNA 2 NPU when the goal is high-throughput, low-latency inference on dense collider-style features? The answer explored here is a forward-only residual MLP whose hidden body is laid out as a conveyor belt across the 32 compute tiles of the Strix Point array. Using the public HIGGS benchmark, we report two operating points: a throughput-oriented $H = 32$, $L = 8$ configuration that sustains about 4.18M samples/s wall-clock with the tiny classifier head left on CPU, and a full-data $H = 64$, $L = 32$ model trained on 10.4M examples that reaches 77.98% test accuracy, 0.865 ROC AUC, and 0.877 PR AUC on the fixed 500k-example test split. The point is not that this model already wins every accuracy leaderboard; it is that the design is small enough to understand end to end, yet strong enough to make a credible systems case for dense event filtering on cheap consumer hardware.

1 Introduction

Modern NPUs become most interesting when they stop being treated as "small GPUs" and start being treated more like conveyor belts. Instead of launching one large shared-memory kernel, the programmer assigns a small local job to one tile, hands the activations to the next tile, and repeats this until the full network has been traversed. That is the intuition behind spatial dataflow hardware. On AMD XDNA 2, the programmer sees a 2D array of compute tiles, each with local SRAM, a VLIW plus SIMD core, a matrix-multiply unit, and explicit routing. The central question is therefore no longer just "what architecture is accurate?", but also "what architecture maps cleanly to many tiny memories connected by explicit streams?".

The focus here is deliberately narrow: a residual MLP for HIGGS, with one residual matrix per tile and a forward-only conveyor-belt deployment on the NPU. The aim is not to survey every model family or every use of XDNA 2, but to isolate one design that can be understood end to end.

HIGGS is a natural benchmark for this study. It is a dense 28-feature binary classification task derived from collider events, introduced by Baldi, Sadowski, and Whiteson as a large-scale signal-versus-background benchmark with roughly 11 million examples [6, 7]. That makes a residual MLP structurally plausible before any hardware argument is made. It also makes the throughput story easier to defend. For example, the CMS Phase-2 Level-1 trigger must inspect detector data at the 40 MHz bunch-crossing rate, produce decisions with a latency budget of about 12.5 μs, and emit accepted events at roughly 750 kHz [9]. This project is not a production trigger design, but it does live in the same systems regime: dense features, strict latency sensitivity, and strong pressure for very high sustained throughput.

2 The hardware model we program against

Figure 1 shows the physical layout targeted here. The Ryzen AI 9 HX 370 exposes 32 compute tiles arranged as 8 columns by 4 compute rows. Below them sit 8 MemTiles, and below those 8 shim tiles that connect the array to host-visible memory and runtime control.

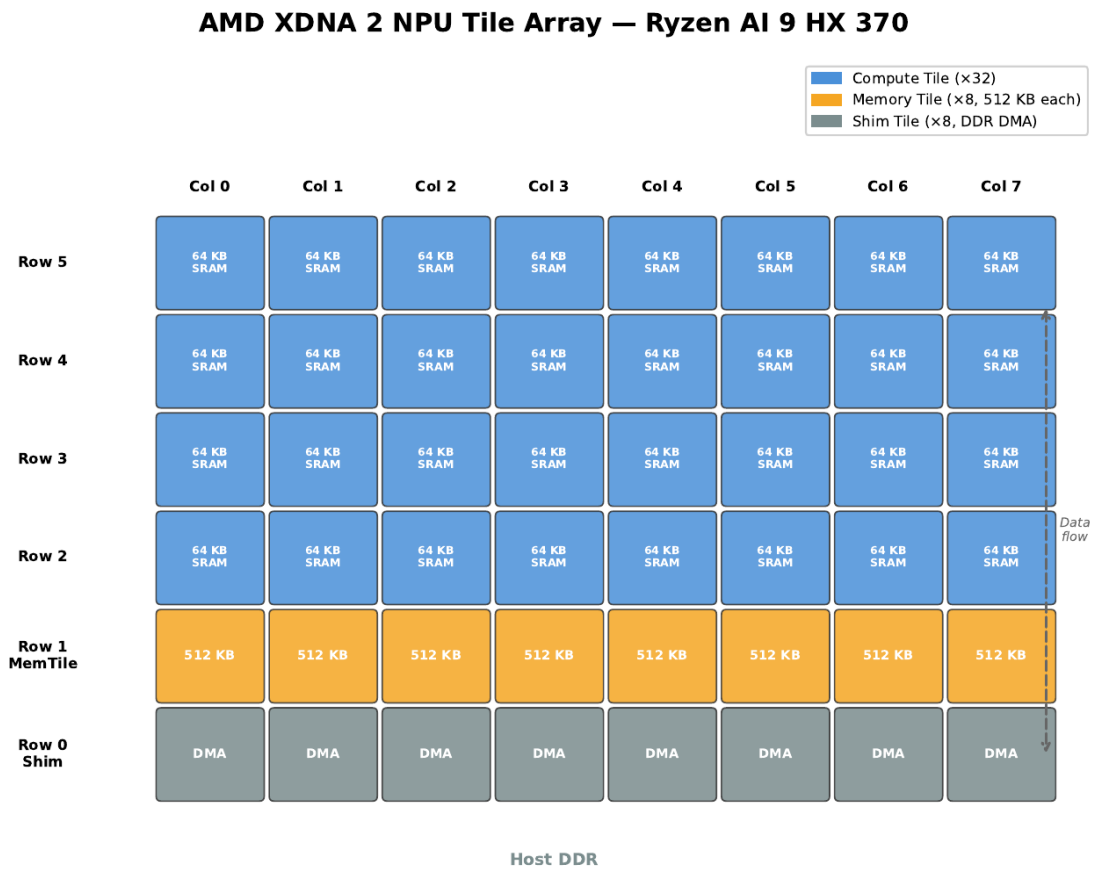

Figure 1: Physical layout of the AMD XDNA 2 NPU used here. The design studied in this paper can use up to all 32 compute tiles (8 columns $\times$ 4 rows).

These constraints shape the model directly. The hidden body is kept square so that each tile stores one $H \times H$ residual weight matrix. Widths are kept as multiples of 8 to match the AIE MMUL datapath. The host only moves the data that are still cheaper to keep on CPU today: the input embed and, for the best current throughput point, the tiny final $H \to 2$ classifier head.

3 A residual network that matches the array

The network studied here is intentionally minimal.

Notation

We use H for hidden width, L for the number of residual blocks, and B for batch size. In the deployment path studied here, one residual block maps to one compute tile.

Model definition

$$h_{\ell+1} = \mathrm{ReLU}(h_\ell W_\ell) + h_\ell, \qquad \ell = 0, \ldots, L-1.$$

Each residual matrix W_ℓ is assigned to one compute tile. Activations enter the array, flow through the tiles in a snake, and return to the host only after the full hidden body has completed. In the current deployment path:

- the host CPU runs the input projection from the native 28 HIGGS features to hidden width H,

Property	Why it matters here
32 compute tiles (8 columns × 4 rows)	Depth maps naturally to tile count; an $L = 8$ body uses 2 columns, an $L = 32$ body uses the full array.
About 64 KB local SRAM per compute tile	Parameters must stay small enough to live locally; that strongly constrains hidden width.
8×8 MMUL-friendly blocked execution	Widths and batch sizes should be divisible by 8, and host-side packing must respect tiled layouts.
Explicit FIFOs, DMA, and routing	Data movement is part of the design, not an afterthought hidden behind a cache hierarchy.
IRON / MLIR-AIE / XRT software stack	The implementation programs the array close to the hardware model rather than through a high-level opaque runtime.

Table 1: Only a few hardware facts matter for this paper. The most important is not the marketing TOPS number, but the combination of tiny tile memories and explicit data movement.

- the NPU runs the residual body,
- the host CPU currently keeps the tiny final $H \to 2$ head when that is faster than the available fused on-array tail.

This design leads to two practical operating points.

- **Throughput point:** $H = 32$, $L = 8$, mapped to 2 columns. This is the fastest end-to-end HIGGS point measured so far.
- **Accuracy point:** $H = 64$, $L = 32$, trained on the full public dataset and mapped to the full 8-column array when the deeper body is streamed on the NPU.

The important design choice is not just "use an MLP", but "use an MLP whose depth and width map almost literally to the array". That keeps the design simple enough to analyze and direct enough to deploy.

4 From Python model to NPU artifact

The toolchain is deliberately short. We start from `aie_kernels/matmul_relu_skip.cc`, which defines the tile-local residual kernel. We then use `resmlp/streaming_design.py` to generate the conveyor-belt MLIR-AIE program for a chosen (H, B, L) shape. `resmlp/streaming_op.py` wraps the compiled XCLBIN and instruction binary as a resident XRT operator. Finally, `resmlp/streaming_infer.py` drives trained checkpoints through that operator and measures wall-clock throughput.

A key detail of the current forward path is that the residual weights are embedded at compile time. Each throughput run therefore builds a checkpoint-specific streaming artifact. That is not the most general deployment contract. It is, however, the cleanest way to study the geometry, routing, and wall-clock behavior of a resident conveyor-belt design.

5 Training protocol on HIGGS

The data pipeline uses the public `jxie/higgs` mirror of the published train/test splits [8]. The local cache materializer produces a fixed 10.5M-example train split and a fixed 500k-example test split. For model selection, training holds out 100k examples from the train split. That leaves 10.4M examples for optimization. All reported runs use the native 28-dimensional feature vector without any padded-width preprocessing. Hyperparameter search and checkpoint selection use held-out validation accuracy. The 500k-example test split is used only for final evaluation of already-selected models.

Model	Training recipe
$H = 32$, $L = 8$ full-data pilot	5 epochs on the AMD Radeon 890M ROCm path; AdamW, batch 8192, cosine $3 \times 10^{-3} \rightarrow 10^{-4}$, weight decay 10^{-4}, no label smoothing.
$H = 32$, $L = 32$ long schedule	20 epochs on the AMD Radeon 890M ROCm path; AdamW, batch 8192, cosine $3 \times 10^{-3} \rightarrow 10^{-4}$, weight decay 10^{-4}, no label smoothing.
$H = 64$, $L = 32$ validation-selected tuning run	Best checkpoint at epoch 46 of a 50-epoch MLflow/Optuna sweep on the AMD Radeon 890M ROCm path; AdamW, batch 4096, cosine $1.069 \times 10^{-3} \rightarrow 8.70 \times 10^{-5}$, weight decay 3.23×10^{-3}, label smoothing 0.0197.

Table 2: Training recipes for the reported HIGGS checkpoints. Both direct runs and validation-driven hyperparameter search were used to produce the reported models.

Model	Test acc.	ROC AUC	PR AUC	Log loss
$H = 32$, $L = 8$ full-data (5 epochs)	75.89%	0.8430	0.8559	0.4861
$H = 32$, $L = 32$ full-data (5 epochs)	76.56%	0.8502	0.8628	0.4762
$H = 32$, $L = 32$ full-data (20 epochs)	76.98%	0.8542	0.8668	0.4705
Validation-selected tuning run: $H = 64$, $L = 32$	77.98%	0.8653	0.8770	0.4543

Table 3: Full-data HIGGS results on the public `jxie/higgs` mirror. The validation split holds out 100k examples from the 10.5M published train split, leaving 10.4M examples for optimization and a fixed 500k-example test set for final reporting.

HIGGS is also a good benchmark because it sits at the intersection of two real literatures. In high-energy physics it is a canonical dense-event benchmark [6]. In tabular machine learning it is one of the standard large-data problems on which tree ensembles such as XGBoost, LightGBM, and CatBoost remain strong baselines, while modern deep tabular models are now more competitive but not universally dominant [10, 11, 12, 13, 14]. That means good-enough accuracy on HIGGS is genuinely worth caring about, even if the main contribution of this work is still systems-oriented.

6 Results

6.1 Accuracy on the full public HIGGS split

Table 3 summarizes the current full-data results. The key point is that the systems design is not accuracy-indifferent. Even this deliberately hardware-friendly residual MLP becomes a credible classifier once it is trained on the full public HIGGS data.

Two observations matter. First, depth helps. Moving from $L = 8$ to $L = 32$ improves every metric even before any serious hyperparameter search. Second, validation-driven hyperparameter tuning pushes the result from the high-0.85 ROC band to 0.865 ROC AUC. It also improves accuracy, PR AUC, and log loss. This is still not a claim of state-of-the-art HIGGS accuracy. It is, however, strong enough that the main contribution of the paper is no longer accuracy-indifferent.

6.2 Throughput of the conveyor-belt deployment

Table 4 reports the current repeated-sample throughput measurements over 50M processed samples. "Wall throughput" includes the host embed, host head, host-visible runtime overheads, and the NPU residual stage. "Kernel throughput" counts only the NPU kernel wait time.

This table sharpens the systems conclusion. The best current wall-clock point is not the deepest model, and it is not the version with the entire graph moved onto the array. On the

Configuration	Head placement	Wall throughput	Kernel throughput
$H = 32$, $L = 8$, 2 columns	CPU head	4.18M samples/s	26.29M samples/s
$H = 32$, $L = 8$, 2 columns	NPU head	3.65M samples/s	7.47M samples/s
$H = 32$, $L = 32$, 8 columns	CPU head	3.49M samples/s	19.06M samples/s
$H = 32$, $L = 32$, 8 columns	NPU head	3.15M samples/s	6.68M samples/s

Table 4: Repeated-sample throughput on the real HIGGS checkpoints. The current winner remains the 8-layer, 2-column path with the classifier head left on CPU.

present tail kernel, the fused $H \to 2$ NPU head is slower than keeping that tiny classifier on CPU. That is an important honest result: the right abstraction boundary in the current implementation is a CPU embed, NPU residual body, and CPU head. The next hardware step is therefore not "force everything on-array" but "move the expensive surrounding stages only when they really improve wall time". A plausible reason is that the $H \to 2$ head is simply too small to amortize the extra synchronization, buffer traffic, and launch overheads of the fused tail path. In other words, the additional runtime machinery costs more than the CPU saves on this tiny matrix multiply.

7 Discussion

This study makes a narrow but defensible claim.

- A residual MLP can be shaped to match the AMD XDNA 2 array very directly: one residual matrix per tile, streamed activations, explicit routing, and checkpoint-specific resident weights.
- That design already delivers multi-million-sample-per-second inference on a real collider-style dense benchmark using cheap consumer hardware.
- The same design family also reaches nontrivial full-data HIGGS quality, culminating in 77.98% test accuracy and 0.865 ROC AUC on the fixed 500k-example test set.

Just as importantly, the paper avoids claims it does not need. It does not pretend the current model is a production trigger, that the current tail kernel is optimal, or that the current HIGGS result is the final word on tabular accuracy. The point is simpler and cleaner: spatial hardware rewards spatially shaped models, and a small, explicit conveyor-belt design can already hit a useful combination of interpretability, throughput, and real-task credibility.

8 Conclusion and next steps

The main result is straightforward: designing a network for spatial compute on AMD XDNA 2 can produce unusually high throughput on real dense-event classification while remaining simple enough to understand and validate end to end. The model family is deliberately modest, but it maps cleanly to the array and delivers both a strong throughput point and a nontrivial full-data HIGGS accuracy point.

The next steps are correspondingly focused. The highest-leverage systems step is to move the $28 \to 32$ embed on-array so that the surrounding host work shrinks further. The highest-leverage modeling step is to continue exploiting the promising $H = 64$, $L = 32$ region with longer schedules and regularization, rather than broadening the scope of the architecture prematurely.

Disclosure of AI-assisted workflow

AI coding and research assistants, including Claude Opus 4.6, GPT-5.4, and Gemini 3.1 Pro, were used extensively during this project to generate code, propose experiments, summarize

results, review literature, and critique or draft text. The author curated the research direction, selected the final claims and figures, verified the reported results, and takes responsibility for the content of this paper.

References

[1] AMD, *IRON*.
https://github.com/amd/IRON

[2] Xilinx / AMD, *MLIR-AIE programming guide*.
https://github.com/Xilinx/mlir-aie/tree/main/programming_guide

[3] Linux kernel documentation, *AMD NPU / amdxdna*.
https://docs.kernel.org/next/accel/amdxdna/amdnpu.html

[4] AMD, *AI Engine documentation (programming model and architecture)*.
https://docs.amd.com/r/en-US/ug1273-versal-acap-design/AI-Engine

[5] *AMD XDNA overview*.
https://en.wikipedia.org/wiki/AMD_XDNA

[6] P. Baldi, P. Sadowski, and D. Whiteson, *Searching for Exotic Particles in High-Energy Physics with Deep Learning*.
https://arxiv.org/abs/1402.4735

[7] UCI Machine Learning Repository, *HIGGS dataset*.
https://archive.ics.uci.edu/dataset/280/higgs

[8] J. Xie, *HIGGS* (Hugging Face dataset mirror with published benchmark splits).
https://huggingface.co/datasets/jxie/higgs

[9] R. Ardino et al., *Development and demonstration of the CMS Phase-2 Level-1 trigger Data Scouting system*.
https://doi.org/10.1088/1748-0221/21/01/C01024

[10] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*.
https://arxiv.org/abs/1603.02754

[11] G. Ke et al., *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*.
papers.nips.cc/paper/6907-lightgbm

[12] L. Prokhorenkova et al., *CatBoost: Unbiased Boosting with Categorical Features*.
https://arxiv.org/abs/1706.09516

[13] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, *Revisiting Deep Learning Models for Tabular Data*.
https://arxiv.org/abs/2106.11959

[14] L. Grinsztajn, E. Oyallon, and G. Varoquaux, *Why do tree-based models still outperform deep learning on tabular data?*.
https://arxiv.org/abs/2207.08815